

 UNIVERSIDAD DON BOSCO	UNIVERSIDAD DON BOSCO FACULTAD DE INGENIERIA ESCUELA DE COMPUTACIÓN
CICLO 2	Materia: Programación Orientada a Objetos (POO404) Guia de Laboratorio No.1 Tema: Introducción a Java

I. OBJETIVOS.

Que el estudiante:

- Reconozca los aspectos generales de la programación en Java
- Se familiarice con el entorno de programación Java brindado por el IDE IntelliJ IDEA.

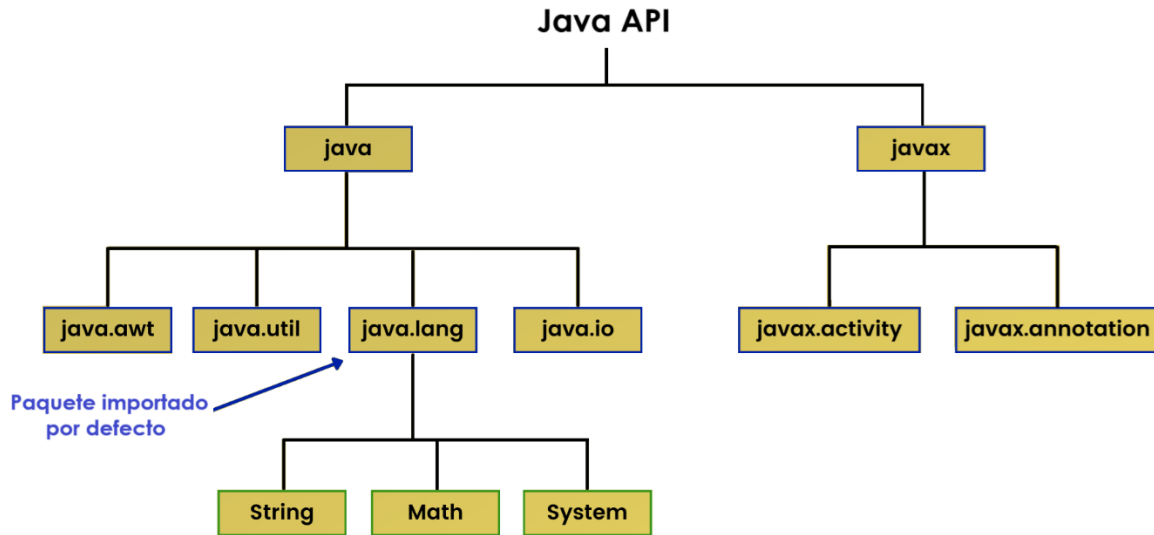
II. INTRODUCCIÓN.

API de Java

Las numerosas clases predefinidas de Java se agrupan en categorías de clases relacionadas, conocidas como paquetes.

Un paquete es una colección de clases, los cuales se conocen colectivamente como la **biblioteca de clases de Java**, o la **interfaz de programación de aplicaciones de Java** [**API** (Por sus siglas en inglés) de Java]. Los paquetes del API de Java se dividen en básicos y opcionales. Los nombres de la mayoría de los paquetes del API de Java comienzan, ya sea con “Java” (paquetes básicos) o “Javax” (paquetes opcionales. Aunque algunos nombres de paquetes en el API de Java comienzan con org.). Muchos de ellos se incluyen como parte del Kit de desarrollo de software para Java 2.

A medida que Java continúa evolucionando, la mayoría de los nuevos paquetes se desarrollan como paquetes opcionales. Estos a menudo pueden descargarse de java.sun.com y se utilizan para mejorar las capacidades de Java.



Paquetes de la API de Java

¿Qué es IntelliJ IDEA?

¹IntelliJ IDEA es un entorno de desarrollo integrado (IDE) para el desarrollo de programas informáticos. Es desarrollado por JetBrains (anteriormente conocido como IntelliJ), y está disponible en dos ediciones: edición para la comunidad y edición comercial.



IntelliJ IDEA

Un IDE es también conocido como una plataforma para desarrollo de diferentes tipos de sistemas, particularmente es una herramienta que facilita la tarea de cualquier desarrollador y sin duda entre más opciones se brinden para la creación de software, mucho más beneficioso es para el programador.

Es necesario también saber que IntelliJ IDEA es un software completamente de paga. Por lo cual su versión para la comunidad está hecha para aplicaciones menos pesadas y empresariales.

¹ Texto obtenido desde: https://es.wikipedia.org/wiki/IntelliJ_IDEA

¿Qué es JetBrains?

JetBrains es un proveedor de software de vanguardia especializado en la creación de herramientas de desarrollo inteligentes, cuyas herramientas están dirigidas a desarrolladores de software y gerentes de proyectos.

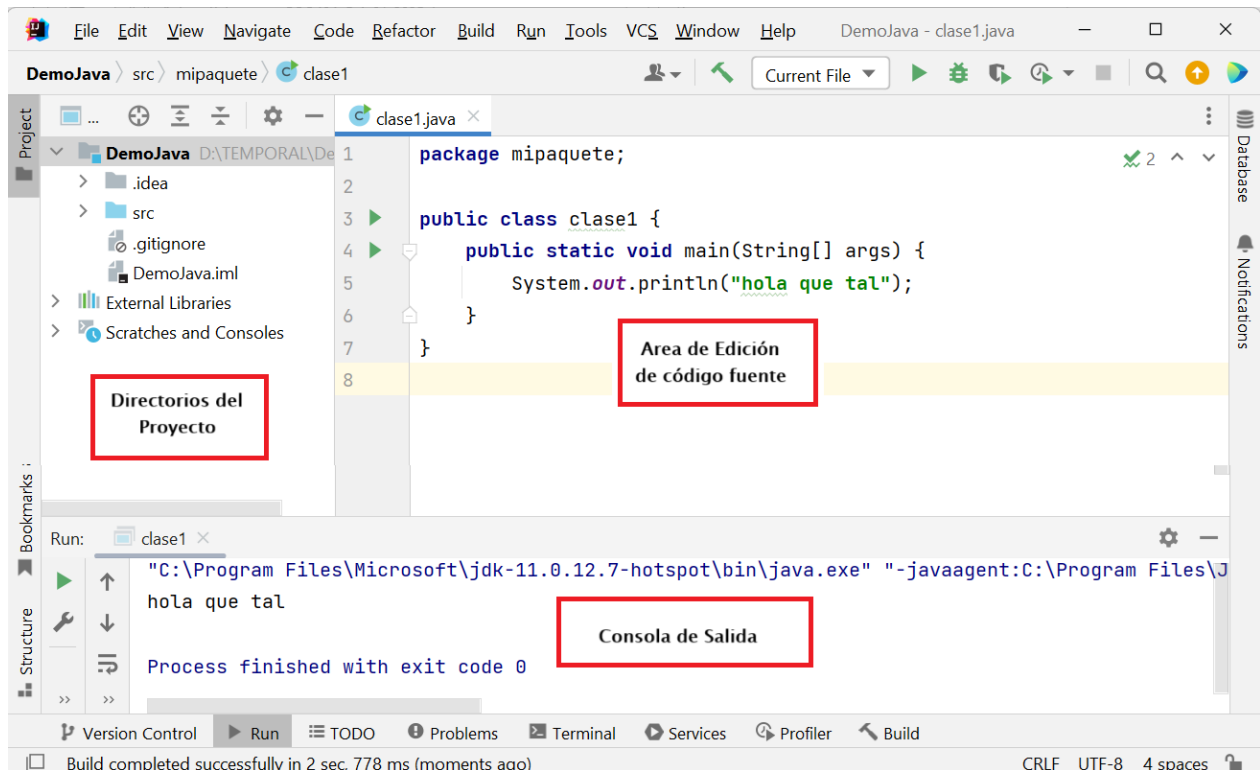


Imagen 1: Vista del Entorno de trabajo de IntelliJ-IDEA

Historia

JetBrains, inicialmente llamado IntelliJ Software, fue fundado en 2000 en Praga por tres desarrolladores de software:13 Sergey Dmitriev, Valentin Kipyatkov y Eugene Belyaev.

El primer producto de la compañía fue IntelliJ Renamer, una herramienta para la refactorización de código en Java. En 2012, después de haber sido el CEO de la compañía durante 12 años, Sergey Dmitriev confió la compañía a dos CEO recién nombrados, Oleg Stepanov y Maxim Shafirov, y se dedicó a sus esfuerzos científicos en el campo de la bioinformática.

En la actualidad siguen desarrollando productos para el desarrollo de software, no solo enfocado en java, sino en otros lenguajes como Kotlin, Groovy y Scala.

Instalar IntelliJ IDEA

La instalación de este entorno de desarrollo es muy sencilla, pero es necesario tomar en cuenta algunos aspectos importantes para este curso; primeramente, este software es de paga por lo cual se requiere de una licencia de producto para poder tener todos los beneficios que ofrece. Como primer paso se debe ir al sitio oficial y registrar una cuenta estudiantil. Vaya al Curso de esta clase en el Aula Digital UDB, para descargar los documentos PDF proporcionados para realizar el registro y obtener el ID de activación para la licencia profesional.

Ahora que cuenta con una cuenta y una licencia activa, puede instalar el IDE IntelliJ IDEA. En la mayoría de los casos, basta con abrir el ejecutable y seguir los pasos que se le presentan, descargado en una unidad de disco con espacio libre suficiente.

Para ejecutar IntelliJ IDEA en un SO Windows, necesitará cumplir los requisitos técnicos detallados a continuación.

Version de 64 bits de Windows 10 o Windows Server 2019 (o versiones posteriores).	
2 GB de RAM	3.5 GB espacio de almacenamiento, recomendado SSD
Resolucion de pantalla: 1024x768 (recomendado)	

Fundamentos de un entorno típico en Java

La siguiente explicación define los pasos típicos para crear y ejecutar un programa en Java, utilizando los entornos de desarrollo de Java. Estos pasos se resumen en la Figura 1 y se explican a continuación

Por lo general, los programas en Java pasan a través de cinco fases para poder ejecutarse como se muestra en la Figura 1. Estas fases son: **Edición, Compilación, Carga, Verificación y Ejecución.**

fase 1 (Edición): Consiste en editar el código fuente de Java en un archivo, mediante un programa de edición (el block de notas de Windows, el vim de Linux, o el que más te guste). Cuando un programador determina que el programa está listo, guarda el programa en un archivo con la extensión **.java**.

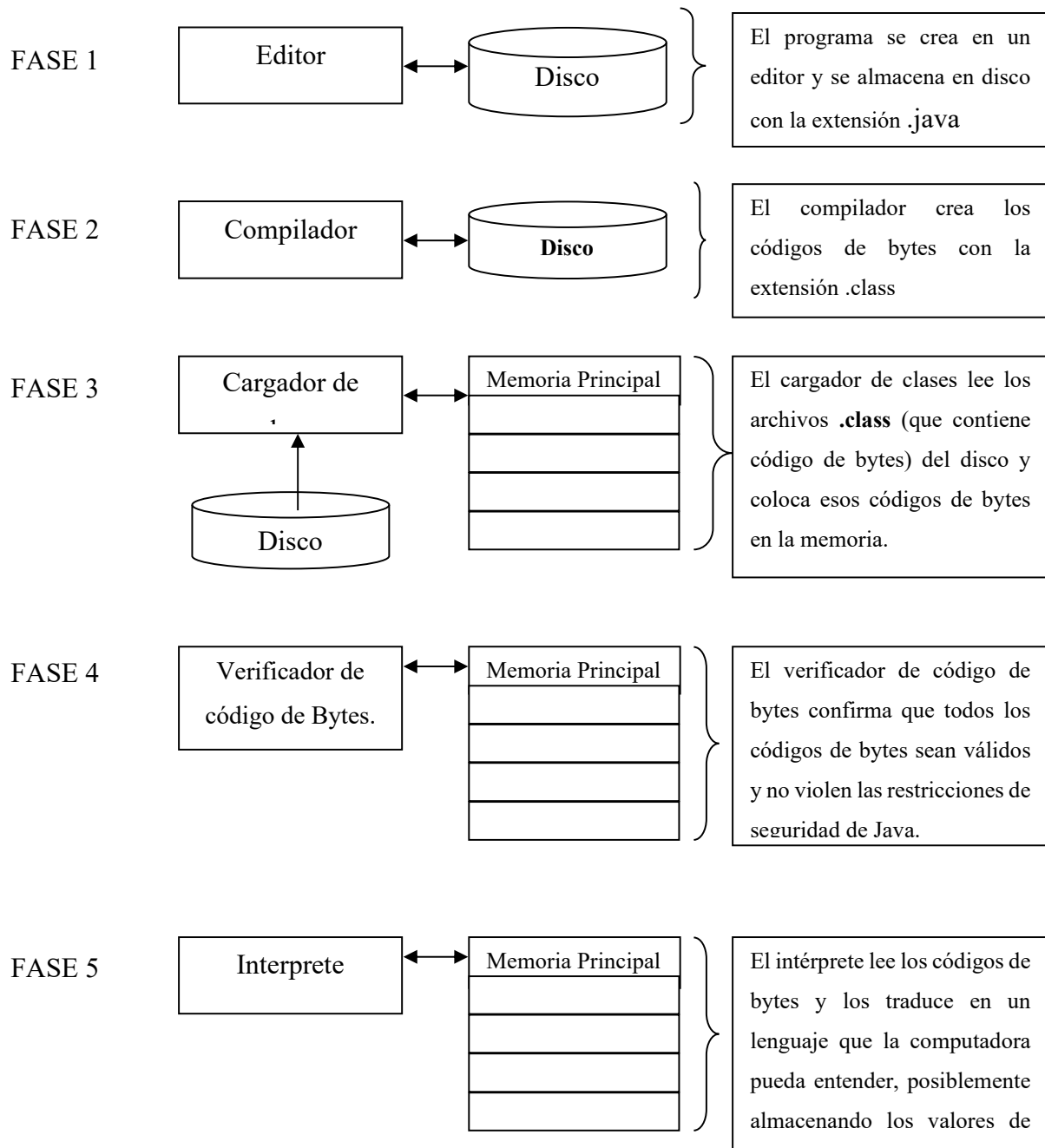


Figura 1: Entorno típico en Java.

fase 2 (Compilacion): El programador proporciona el comando **javac** para compilar el programa. El compilador de Java traduce el programa a código de bytes (bytecode), con las instrucciones que el intérprete de Java (o máquina virtual) puede entender. Por ejemplo, para compilar un programa llamado **Hola.java**, escriba:

javac Hola.java

En la ventana de comandos de su sistema. Si el programa se compila correctamente, el compilador genera un archivo **Hola.class**. Este archivo contiene códigos de bytes que se interpretan en la fase de ejecución.

fase 3 (Carga): Se conoce como carga. El archivo **.class** con el bytecode debe colocarse en memoria antes de ejecutarse. De esto se encarga el *cargador de clases*, que carga el archivo **.class** (desde un disco o a través de una red) y los transfiere a la memoria principal.

A medida que se cargan las clases, el verificador de códigos de bytes se encarga de verificar sus códigos de bytes.

fase 4: Este proceso de verificación asegura que los códigos de bytes de las clases sean válidos y que no violen las restricciones de seguridad de Java. Java implementa una estrecha seguridad, ya que los programas en Java que llegan a través de la red no deben ser capaces de dañar sus archivos o su sistema (como podrían hacerlo un virus de computadora). Tome en cuenta que la verificación del código de bytes también se lleva a cabo en aplicaciones que descargan clases a través de una red.

fase 5 (Ejecucion): el intérprete, bajo el control del sistema operativo, interpreta el programa un código de bytes a la vez, realizando en esta forma las acciones especificadas por el programa.

Existen dos tipos de programas para los cuales ocurre este proceso: **las aplicaciones y los applets**.

1. **Aplicación:** es un programa (como por ejemplo un programa de procesamiento de palabras, de hojas de cálculo, de dibujo o de correo electrónico) que generalmente se guarda y ejecuta desde el equipo local de usuario.
2. **Applet:** es un pequeño programa que se guarda en un equipo remoto, al cual se conectan los usuarios a través de un navegador Web. El equipo remoto se conoce como un servidor Web.

Las **aplicaciones** se cargan en memoria y se ejecutan, utilizando el intérprete de Java mediante el comando `java`. Al ejecutar una aplicación de Java llamada, por ejemplo, `Hola.class`, el comando a ejecutar sería:

java Hola

Este invoca al intérprete para la aplicación Hola, y hace que el cargador de clases cargue la información utilizada en el programa Hola. [Nota: El intérprete de Java se conoce también como la Máquina Virtual, o la **JVM (Java Virtual Machine)**].

Por otra parte, los **applets** se cargan desde un equipo remoto en el navegador, se ejecutan en éste y se descartan cuando termina la ejecución.

Para ejecutar un applet de nuevo, el usuario debe dirigir un navegador a la ubicación apropiada en Internet y volver a cargar el programa en el navegador.

Los navegadores Web como el **FireFox o Internet Explorer** se utilizan para ver los documentos en Internet, que generalmente son archivos HTML. Un documento HTML puede contener un applet de Java. Cuando el navegador ve la referencia (las etiquetas) a un applet en un documento HTML, inicia el cargador de clases de Java para cargar el applet. Todos los navegadores que cuentan con soporte para Java tienen un intérprete de Java incluido (es decir, una JVM). Una vez que se carga el applet, el intérprete de Java lo ejecuta. Los applets también pueden ejecutarse desde una línea de comandos, mediante el comando **appletviewer** que se incluye con el J2SDK. Al igual que un navegador, el **appletviewer** requiere un documento HTML para poder invocar al applet. Por ejemplo, si el archivo Hola.html hace referencia al applet Hola, el comando **appletviewer** se utiliza de la siguiente manera:

appletviewer Hola.html

Este comando ocasiona que el cargador de clases cargue la información utilizada en el applet Hola. El **appletviewer** es un navegador muy básico; solo sabe cómo interpretar referencias a los applets e ignora el resto de código HTML del documento.

Nota: el compilador de Java, `javac`, no es un compilador tradicional en cuanto a que no convierte un programa de Java, de código fuente a código máquina nativo para una plataforma computacional específica. En vez de ello, el compilador de Java traduce el código fuente en código de bytes; el lenguaje de la Máquina Virtual de Java (JVM). La JVM es un programa que simula la operación de una computadora y ejecuta su propio lenguaje máquina (es decir, código de bytes de Java).

III. Procedimiento.

PARTE 1: Reconociendo el Entorno de IntelliJ-IDEA

1. Ingrese al menú de Inicio de Windows y escriba *Intell*. Cuando en las sugerencias de programas, aparezca el nombre IDE IntelliJ-IDEA, seleccionala con el ratón. Se mostrara por unos segundos a la pantalla de presentación de la imagen 2.



Imagen 2: Pantalla inicial de IntelliJ IDEA 2024.1

2. Luego, se abre una ventana manera similar a la de la Imagen 3, que permite abrir un proyecto ya existente o crear un proyecto.

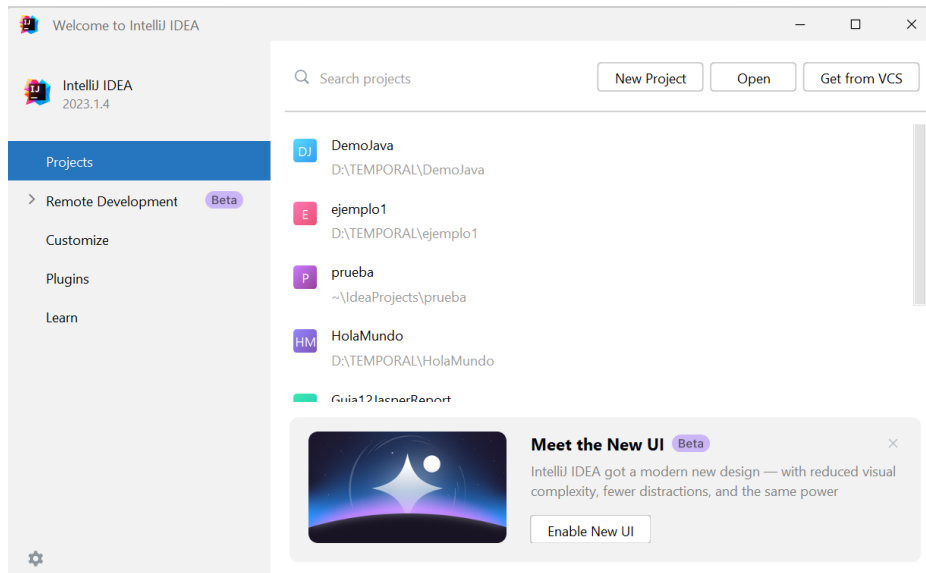


Imagen 3: Ventana de bienvenida de IJ-IDEA

Importante: Si se abre una ventana similar a la mostrada en la Imagen 1 (ver la introducción teórica de esta practica), significa que ya esta abierto un Proyecto específico. En ese caso, usted debe elegir del menú principal a las opción *File -> Close Project*

3. Para crear su primer proyecto, de la ventana de la Imagen 3, haga clic en el botón *New Project*.
4. En la ventana de creación de proyecto, tal como se muestran en la Imagen 4, defina como nombre a **POOGuia1** y en Location, ubique directamente a la ruta C:\. Además, en el combo JDK, seleccione una versión instalada igual o similar a la mostrada en imagen.

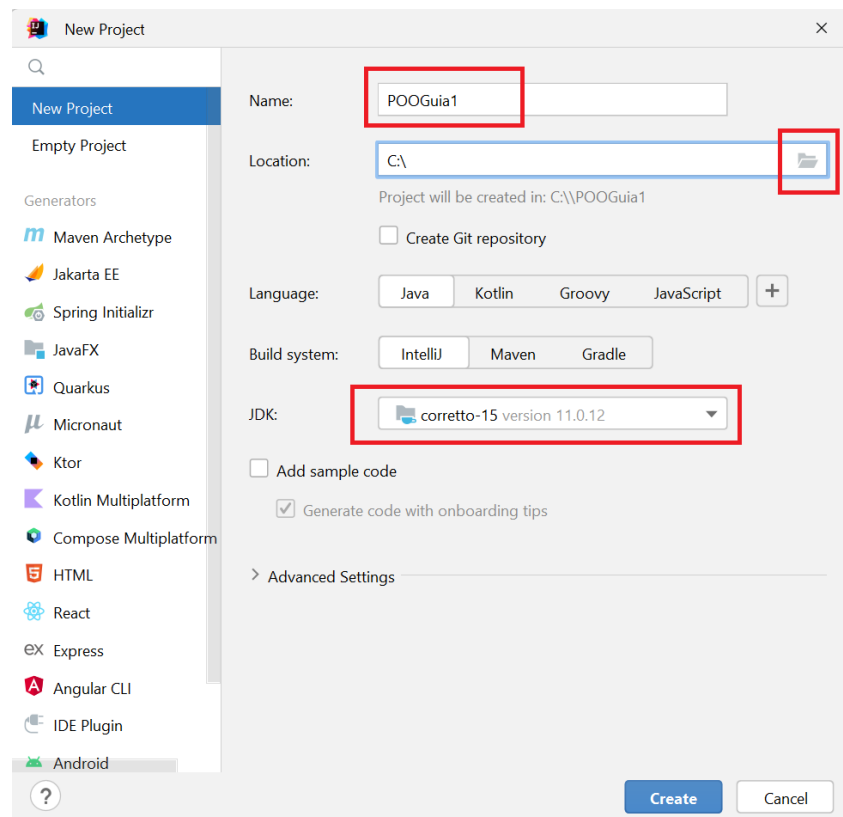


Imagen 4: campos a definir para su Proyecto

5. Haga clic en botón **Create**. Se abra una ventana (similar a la Imagen 1) con los archivos y carpetas que componen a su nuevo proyecto.

Opcional: Observe que, en las imágenes de ventanas del procedimiento de esta guía, se usa el formato claro/brillante (no el oscuro predeterminado que ofrece IJ-IDEA). Si usted lo desea, cambie el estilo oscuro de IJ-IDEA al claro, localizando del menú principal a *File -> Setting*. Ubica el campo **Theme** y elija *IntelliJ Light*. Además, para una lectura mas cómoda, puede cambiar el Zoom a un 125%. Y finalmente, hace clic en botón OK

6. Como siguiente paso, creara un paquete, en donde se guardarán todas las clases Java de su proyecto.

Del *explorador de proyecto* (ventana a la izquierda), localice y haga clic secundario sobre el directorio **src**. Del menú contextual, seleccione **New** y en **Package**.

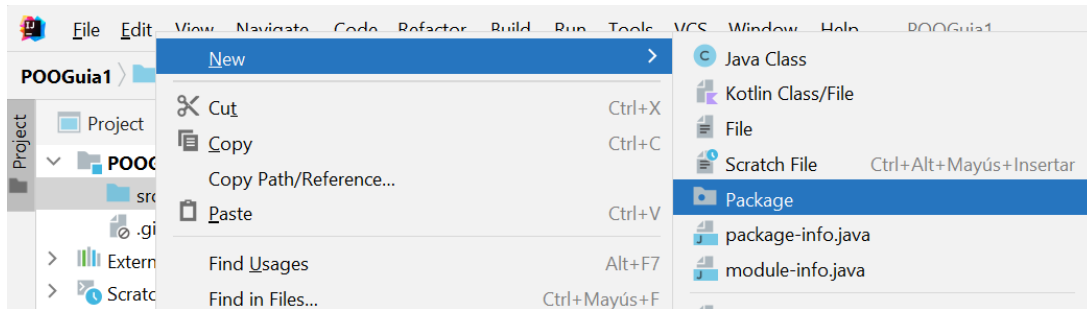


Imagen 5: Pasos para crear un paquete

7. Asigne el nombre **Ejercicio1** a su nuevo paquete y confirme haciendo *Enter* para crearlo.

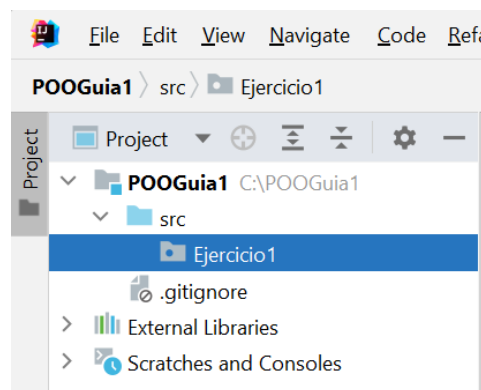


Imagen 6: paquete creado en carpeta src del proyecto

8. Ahora creara una Clase dentro del paquete creado. Para ello, repita el paso anterior, pero en lugar de elegir **Package**, seleccione **Java Class**. Asigne **Ejemplo1** como nombre a la Clase y presione *Enter*.

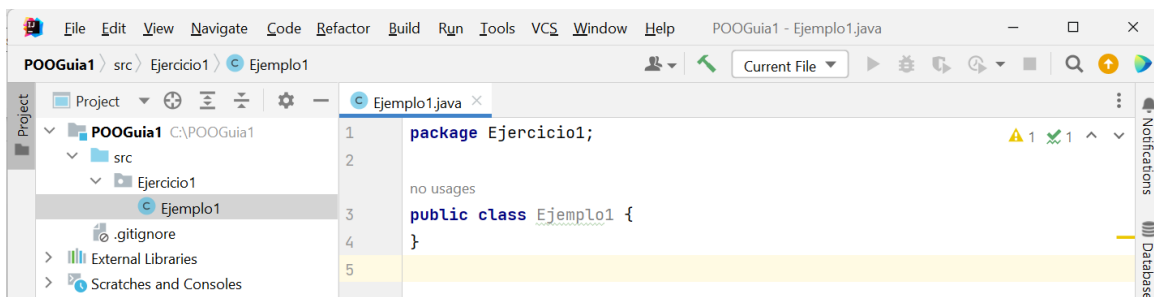


Imagen 7: Ventana para edición de código de la clase Ejemplo1

9. Se abre la ventana para edición del código de clase, con la definición preliminar, que incluye la referencia al paquete **Ejercicio1**, donde esta almacenado el archivo con la clase **Ejercicio1**. Y el encabezado de la clase **Ejemplo1**.

Acerca de este código:

- La palabra clave **class** introduce una declaración de clase en Java, la cual debe ir seguida inmediatamente por el nombre de la clase.
 - **Por convención, todos los nombres de clases en Java comienzan con una letra mayúscula, y la primera letra de cada palabra en el nombre de la clase debe ir en mayúscula** (por ejemplo, **EjemploDeNombreDeClase**). El nombre de la clase se conoce como identificador, y está compuesto por una serie de caracteres que pueden ser letras, dígitos, guiones bajos (**_**) y signos de moneda (**\$**); sin embargo, no puede comenzar con un dígito ni tener espacios.
 - Todas las extensiones de clases en Java se guardan en archivos que terminan con la extensión **“.java”**.
 - Cuando usted guarda su declaración de Clase como **public** en un archivo, el nombre de éste debe ser el mismo nombre de la Clase, seguido de la extensión **“.java”**. Para el caso de su código hasta ahora, el nombre del archivo **Ejemplo1.java**, debe coincidir con el nombre de la clase **Ejemplo1** definida en su interior.
 - **Error común de programación:** *Java es sensible a mayúsculas y minúsculas*. El no utilizar la combinación apropiada de letras minúsculas y mayúsculas para un identificador, generalmente produce un error de compilación.
10. Para iniciar el contenido de la clase, se requiere definir un método de inicio llamado **main**, el cual sera publico y de tipo estático.

Para ello ubique el cursor justo después de la llave de apertura (**{**) y presione *Enter*. En la línea en blanco, escriba **main**. Cuando el editor le ofrezca definir al método **main** (**main()** **method declaration**), presione la tecla **TAB**. Se generara la definición del método **main**, así:

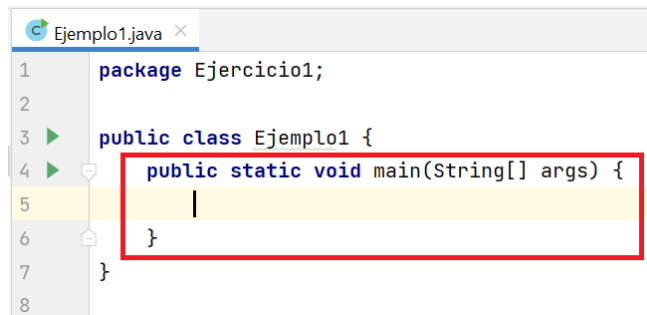


Imagen 8: creación del método estático main.

Importante:

- En una aplicación en Java, cada Clase puede contener todos los métodos que se requieran, pero solamente uno de esos métodos deberá llamarse **main**, ya que el interprete de java iniciara la ejecución solamente desde este metodo **main**. Si no se define, el intérprete de java no ejecutará la aplicación.

11. Antes de la definición de main, incluya una línea vacía y escriba:

```
//definición del método de inicio de ejecución del programa
```

Esta porción de código indica un comentario de línea al estilo de c++. El compilador de Java ignora estos comentarios.

Los programadores insertan comentarios para documentar los programas y mejorar la legibilidad de estos. Los comentarios también ayudan a otras personas a leer y comprender un programa.

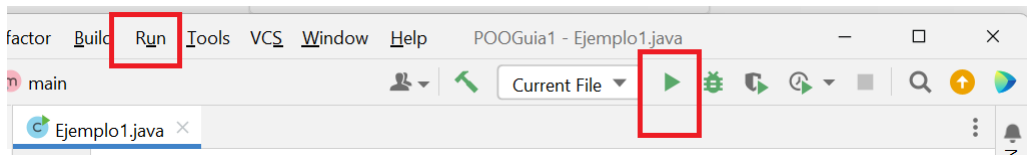
12. Dentro de este método **main**, digite la siguiente línea de código.

```
System.out.println("Bienvenido al IDE de IntelliJ IDEA");
```

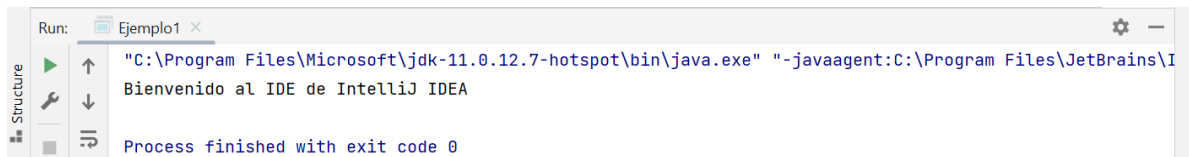
Importante:

- **System.out** se conoce como el objeto de salida estándar. **System.out** permite a las aplicaciones en Java mostrar conjuntos de caracteres en la ventana de comandos, desde la cual se ejecuta la aplicación en Java.
- Una cadena de caracteres debe ir limitada entre los caracteres de comillas dobles. A una cadena también se le denomina cadena de caracteres, mensaje o literal de cadena. Genéricamente, nos referimos a los caracteres entre comillas dobles como cadenas. El compilador no ignora los caracteres de espacio en blanco dentro de las cadenas.

13. Proceda a ejecutar su aplicación Java. Localice el botón Run (ver imagen a continuacion), en la parte superior de la ventana o también, desde el menú principal, seleccione *Run* -> *Ejemplo1.java* o también, presione la combinación de teclas Shift + F10.



14. El resultado de la ejecución se vera en una ventana de la parte inferior.



15. Agregue las siguientes líneas de código después de la línea que invoca a `System.out`:

```
System.out.print("");
System.out.print("Soy Ricardo Maldonado y ");
System.out.print("estoy en el Grupo 01L. \n");
```

Reemplace el nombre resaltado por su propio nombre y apellido, así como el grupo al cual esta inscrito a la practica.

La función `print` es casi idéntica a `println`. La diferencia radica en que `print` escribe la cadena en la pantalla y deja el cursor justo después del último carácter impreso. En cambio, `println` imprime un retorno de carro, dejando el cursor al inicio de la próxima línea vacía.

16. Vuelva a ejecutar el programa y observe las diferencias al ejecutar `println` y `print`
17. Guarde los cambios realizados hasta, con la opción del menú *File->Save All* o presionando la combinación de teclas `Ctrl + S`
18. Cierre el proyecto, presionando la opción del menú *Fila -> Close Project*.
Se retorna a la ventana de selección de proyectos recientes de IJ-IDEA

PARTE 2: Lectura de datos por Consola

19. Del listado de proyectos recientes, localice y haga clic sobre su proyecto creado en este procedimiento.

20. Desde la ventana del explorador de proyecto, haga clic secundario sobre la carpeta src y cree otro paquete, con el nombre **Ejercicio2**.
21. Dentro del nuevo paquete, cree una clase Java. Guardela con el nombre **LecturaConsola**.
Evalúe los cambios en la definición de esta nueva clase (a que paquete pertenece, el nombre del archivo .java y el de la clase misma).
22. Justo después de la línea con la declaración package a la cual pertenece la nueva clase, inserte la siguiente línea:

```
import java.util.Scanner;
```

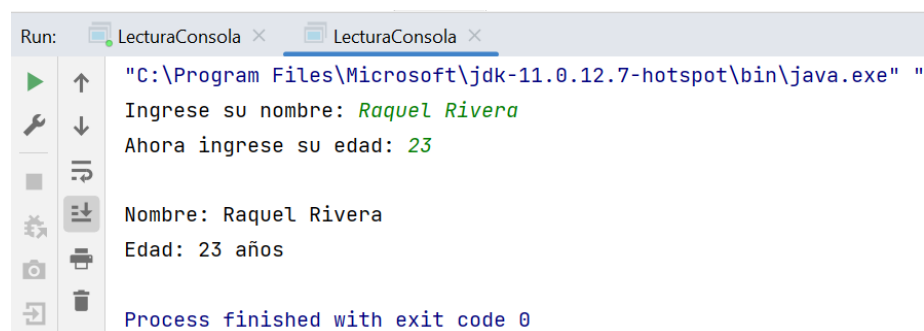
La sentencia import permite utilizar las clases de java incluidas en el paquete util.

23. Dentro de la clase, defina nuevamente a un método de inicio main.

Luego, dentro de la definición de este método, digite el siguiente código:

```
Scanner lector = new Scanner(System.in);  
String nombre = "";  
int edad;  
System.out.print("Ingrese su nombre: ");  
nombre = lector.nextLine();  
System.out.print("Ahora ingrese su edad: ");  
edad = lector.nextInt();  
  
System.out.println("\nNombre: "+nombre);  
System.out.println("Edad: "+edad+" años");
```

24. Proceda a ejecutar la aplicación. Responda al dialogo en la ventana de ejecución, digitando su nombre y presione *Enter*. Ingrese un valor entero como edad y presione *Enter*. El resultado será parecido al siguiente:



```
Run: LecturaConsola x LecturaConsola x  
"C:\Program Files\Microsoft\jdk-11.0.12.7-hotspot\bin\java.exe" "  
Ingrese su nombre: Raquel Rivera  
Ahora ingrese su edad: 23  
  
Nombre: Raquel Rivera  
Edad: 23 años  
  
Process finished with exit code 0
```

Acerca de este código:

- **Uso de clase Scanner:**

- Es una clase incluida en el paquete java.util de Java. Por lo tanto, hay que referenciar/importar a este paquete antes de usar a la clase Scanner, utilizando la palabra import y el paquete que tiene a la clase a utilizar.
- Luego, se declara un objeto (lector en el ejemplo) de esta clase Scanner.
- Al instanciar al objeto, se le envia como argumento al objeto de consola de java (objeto System.in).
- Ahora, para leer lo que el usuario está introduciendo por la consola deberemos de utilizar el método nextLine. Este nos devolverá los caracteres que encuentre en la consola hasta encontrarse un retorno de carro (Enter). El valor se lo asignaremos a una variable String.

```
nombre=reader.nextLine();
```

- El método nextInt lee una cadena de caracteres que representa a un valor entero, el cual es guardado en una variable de tipo int:

```
edad = lector.nextInt();
```

25. Guarde los cambios realizados, presionando la combinación de teclas Ctrl + S

PARTE 3: Utilizacion de Cuadros de Dialogo

Los *Cuadros de Diálogo* son ventanas en las que los programas muestran mensajes importantes a los usuarios del programa.

Uno de los puntos fuertes de Java es su extenso conjunto de clases predefinidas que los programadores pueden usar, en vez de tener que “reinventar la rueda”. A lo largo del contenido de esta materia, emplearemos muchas de esas clases.

En este ejemplo utilizamos la clase predefinida de Java JOptionPane, la cual se encuentra en el paquete javax.swing.

Esta clase de Java proporciona cuadros de diálogo previamente empaquetados, los cuales permiten a los programadores mostrar ventanas que contengan mensajes para los usuarios.

El primer ejercicio, muestra la salida de un programa en la ventana de comandos. Y ahora, en el segundo ejemplo vamos a mostrar la salida de un programa en un cuadro de dialogo.

26. Para continuar, desde el Explorador de su Proyecto, cree un nuevo paquete en la carpeta **src** y guárdelo con el nombre **Ejercicio3**.

27. Dentro de este nuevo paquete, agregue una nueva clase al proyecto, con el nombre de **CuadroDialogo**.

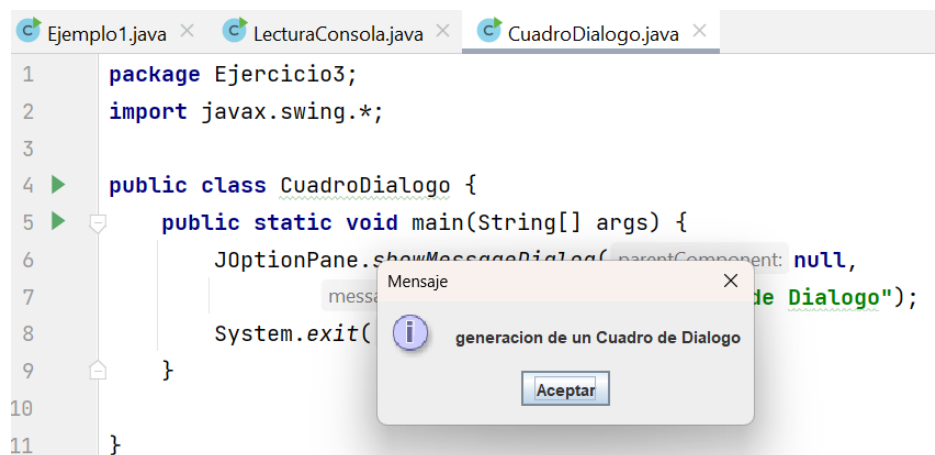
28. Dentro del contenido del archivo CuadroDialogo.java, antes de la definición de la clase, importe el paquete de Java javax.swing, digitando la sentencia:

```
import javax.swing.*;
```

29. Ahora, dentro de la definición de clase, vuelva a crear a un método **main**. Y dentro de su definición, redacte el siguiente código:

```
public static void main(String[] args) {  
    JOptionPane.showMessageDialog(null,  
        "generacion de un Cuadro de Dialogo");  
    System.exit(0);  
}
```

30. Proceda a ejecutar el programa. Obtendra el siguiente resultado:



Acerca de este código:

Sentencia: `import javax.swing.*;`

- Los programadores utilizan declaraciones **import** para identificar las clases predefinidas que se utilizan en un programa en Java. El compilador trata de cerciorarse de que usted utilice correctamente las clases del API de Java.
- Las declaraciones `import` ayudan al compilador a localizar las clases que usted desea utilizar. Por cada nueva clase que utilizamos del API de Java, debemos indicar el paquete en el que se encuentra esa clase. Esta información sobre el paquete es importante, ya que le ayuda a localizar las descripciones de cada paquete y cada clase en la documentación del API de Java.

Importante: todas las declaraciones import deben aparecer antes de la declaración de la clase.

- Olvidar incluir una declaración import para una clase utilizada en su programa, generalmente produce un error de compilación con el mensaje “cannot resolve symbol” (no se puede resolver el símbolo). Cuando esto ocurra, compruebe que haya proporcionado las declaraciones import correctas, y que los nombres en las declaraciones import estén escritos apropiadamente.
- Esta línea indica al compilador que nuestro programa utiliza la clase JOptionPane del paquete javax.swing. Este paquete contiene muchas clases que ayudan a los programadores en Java a crear Interfaces Gráficas de usuario (GUI, por sus siglas en inglés) para las aplicaciones.

Sentencia:

```
JOptionPane.showMessageDialog(null, "generacion de un Cuadro de Dialogo");
```

- Llama al método showMessageDialog de la clase JOptionPane para mostrar un cuadro de diálogo que contiene un mensaje. Este método requiere dos argumentos. Cuando un método requiere varios argumentos, éstos se separan con comas (.). El primer argumento ayuda a la aplicación en Java a determinar en donde se va a colocar el cuadro de dialogo y el segundo argumento es la cadena por mostrar en el cuadro de dialogo. Cuando el primer argumento es null, el cuadro de dialogo aparece en el centro de la pantalla.

Sentencia: `System.exit(0);`

- Utiliza el método estático exit de la clase System para terminar la aplicación. Esto se requiere para terminar cualquier aplicación que muestre una interfaz gráfica de usuario. Observe una vez más la sintaxis utilizada para llamar el método: el nombre de la clase (System), un punto (.) y el nombre del método (exit)

PARTE 4: Ingreso de datos utilizando Cuadros de Dialogo

31. Cree un nuevo paquete, con el nombre **Ejercicio4** y en su interior, cree a una clase java llamada **SumaEnteros**.
32. Al inicio de la definición de esta Clase, utilice la sentencia **import** para importar la clase JOptionPane incluida en el paquete de Java, así:

```
import javax.swing.JOptionPane;
```

33. Dentro de la nueva clase, cree nuevamente al método **main**. Y dentro del método **main**, redacte el siguiente código:

```
//primera y segunda cadena de texto brindada por usuario
String primernumero;
String segundonumero;

int numero1, numero2;
int suma;

primernumero = JOptionPane.showInputDialog("Digite el primer numero");
segundonumero = JOptionPane.showInputDialog("Digite el segundo numero");

numero1 = Integer.parseInt(primernumero);
numero2 = Integer.parseInt(segundonumero);

suma = numero1 + numero2;

JOptionPane.showMessageDialog(null, "La suma es:" +
    suma,"Resultado",JOptionPane.PLAIN_MESSAGE);

System.exit(0);
```

34. Proceda a ejecutar el nuevo programa. Ingrese los datos solicitados por medio de cuadros de dialogo y verifique que el resultado sea correcto.

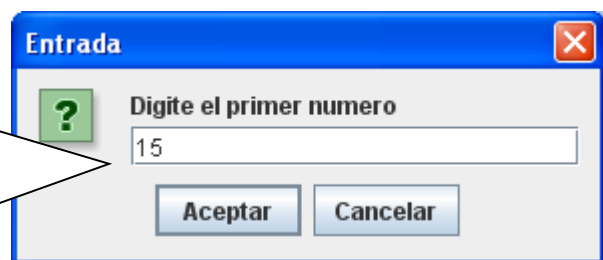
Acerca de este código:

Sentencia:

```
primernumero = JOptionPane.showInputDialog("Digite el primer numero");
```

- Utiliza el método **showInputDialog** de **JOptionPane** para mostrar el cuadro de dialogo. El argumento para **.showInputDialog** indica lo que el usuario debe escribir en el campo de texto. Este mensaje se llama indicador, ya que hace que el usuario realice una acción específica.

Cuando el usuario hace clic en Aceptar, **showInputDialog** devuelve el 15 que escribió el usuario al programa, como un valor **String**. El programa debe convertir el **String** a un entero.



Sentencia:

```
numero1 = Integer.parseInt(primernumero);//conversión del primer números  
numero2 = Integer.parseInt(segundonumero);//conversión del segundo números
```

- Convierten los dos valores **String** que introduce el usuario en valores **int**. El método estático **parseInt** de la clase **Integer** convierte su argumento **String** en número entero. La clase **Integer** se encuentra en el paquete **java.lang**.

Sentencia:

```
JOptionPane.showMessageDialog(null, "La suma es:" +  
suma, "Resultado", JOptionPane.PLAIN_MESSAGE);
```

- Utiliza el método estático **showMessageDialog** de la clase **JOptionPane** para mostrar el resultado de la suma. Esta nueva versión de método **showMessageDialog** de **JOptionPane** requiere 4 argumentos.
- El primer argumento **null** indica que el cuadro de dialogo de mensaje aparecerá en el centro de la pantalla.
- El segundo argumento es el mensaje a mostrar.
- El tercer argumento **“Resultados”** representa la cadena que debe aparecer en la **barra de título** del cuadro de dialogo.
- El cuarto argumento, **JOptionPane.PLAIN_MESSAGE**, es el tipo de cuadro de dialogo, un valor que indica el tipo de cuadro de dialogo de mensaje que se va a mostrar. Este tipo de cuadro de dialogo no muestra un icono a la izquierda del mensaje.
- El argumento de la línea anterior (**"La suma es: " + suma**) utiliza el operador **+** para concatenar en el mensaje de salida la cadena **La suma es:** con el valor de otro tipo (la variable **suma**). La expresión se evalúa de la siguiente manera:
- Java determina que los dos operadores del operador **+** (la cadena **“la suma es”** y el entero **suma**) son de distintos tipos, y que uno de ellos es **String**.
- Java convierte automáticamente **suma** a **String**.
- Java adjunta la representación **String** de **suma** al final de **“la suma es”**.
- El método **showMessageDialog** muestra la cadena resultante en el cuadro de dialogo.

35. Vuelva a ejecutar el programa. Pero esta vez, deje vacío el cuadro de texto que pide el primer número y de clic en Aceptar. Se desencadena un *error lógico en tiempo de ejecución*.

Este error también se desencadena si el usuario escribe un valor no entero o haga clic en el botón Cancelar del cuadro de diálogo.

Más adelante hablaremos sobre el **Manejo de excepciones**, para hacer que sus programas sean más robustos, al permitirles manejar dichos errores. Esto también se conoce como hacer que sus programas sean tolerantes.

PARTE 5: Tomas de decisión en Java, sentencia if

36. Cree un nuevo paquete, con el nombre **Ejercicio5** y una nueva clase dentro de este, con el nombre **Comparacion**.

37. Dentro de la clase Comparacion escriba la siguiente definición:

```
package Ejercicio5;
import javax.swing.JOptionPane;

public class Comparacion {
    public static void main(String[] args) {
        String primernumero, segundonumero, resultado;
        int numero1, numero2;

        primernumero =
            JOptionPane.showInputDialog("Escriba el primer entero:");
        segundonumero =
            JOptionPane.showInputDialog("Escriba el Segundo Entero:");

        numero1= Integer.parseInt(primernumero);
        numero2 = Integer.parseInt(segundonumero);

        resultado= "";
        if (numero1==numero2) {
            resultado= resultado + numero1 + " == " + numero2;
        }
        if (numero1 != numero2) {
            resultado= resultado + numero1 + " != " + numero2;
        }
        if (numero1 < numero2) {
            resultado= resultado + "\n" + numero1 + " < " + numero2;
        }
        if (numero1 > numero2) {
            resultado= resultado + "\n" + numero1 + " > " + numero2;
        }
    }
}
```

```

        if (numero1 <= numero2)
            resultado= resultado + "\n" + numero1 + " <=" + numero2;

        if (numero1 > numero2)
            resultado= resultado + "\n" + numero1 + " >=" + numero2;

        JOptionPane.showConfirmDialog(null, resultado,
            "Resultado de la comparación",JOptionPane.INFORMATION_MESSAGE);
        System.exit(0);
    } //fin metodo main
} //fin clase Comparacion

```

38. Ejecute el programa. Digite 2 numeros enteros y compruebe los resultados de las comparaciones aceptadas entre ambos valores recibidos.

PARTE 6: Lectura de parámetros de consola

39. Cree un nuevo paquete, con el nombre **Ejercicio6** y una nueva clase dentro de este, con el nombre **LecturaParametros**.

40. Dentro de la nueva clase, define al siguiente método main:

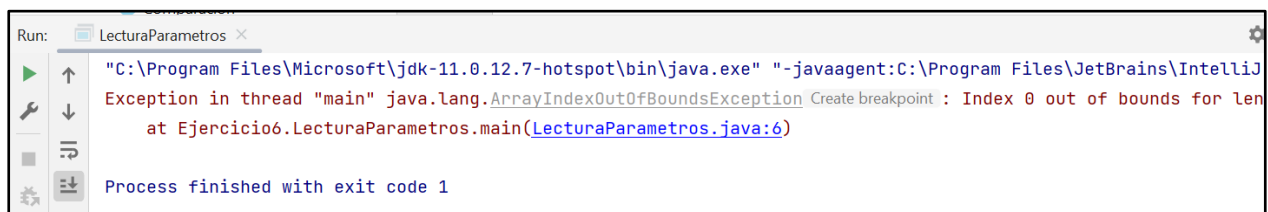
```

package Ejercicio6;

public class LecturaParametros {
    //lectura de datos enviados desde el cursor
    public static void main(String[] args) {
        System.out.println("Parametro 1: "+args[0]);
        System.out.println("Parametro 2: "+args[1]);
    }
}

```

41. Ejecutar la clase y verificar si da el siguiente error.



Analice e intente darle una explicación a su docente acerca del error y solicitarle como se deberá de hacer para ejecutar correctamente a esta clase.

42. Guarde los cambios realizados y cierre su proyecto Java.

V. EJERCICIOS COMPLEMENTARIOS.

1. **(25%)** Modifique el Ejercicio 4, para que no permita que el usuario ingresar números negativos.

Si acaso los ingresará, debe notifique por medio de una ventana que no se pueden ingresar números negativos y finalizara la ejecución (sin operar a valores recibidos).

2. **(35%)** Un estudiante de electrónica requiere una aplicación en Java que calcule el valor del capacitor equivalente (tanto en serie como en paralelo) de un conjunto 3 de capacitores. Los capacitores que se requieren se miden en microfaradios.

Restricciones de diseño:

- Clase implementada para la solución no debe incluir campos, solamente define métodos de acceso estático. Cada campo declarado ocasionará un descuento del 30% en la nota de la solución.
 - Los datos de entrada se deben solicitar solamente con cuadros de dialogo generados con JOptionPanel.
 - Los resultados se mostrarán en la consola de comandos. Estos incluyen los valores de los capacitores recibidos (para confirmar que fueron ingresados correctamente) y finalmente, ver los valores de salida solicitados.
 - En caso de que usuario no brinde un dato de entrada o su valor sea cero o negativo, se le debe informar que no ingreso un valor apropiado y que este programa se finalizara en ese momento (sin realizar cálculos ni ver resultados).
 - El cálculo del *capacitor equivalente en conexión serie* debe ser realizado por un método distinto a **main**. Este nuevo método sera estático, recibe los valores de los capacitores en parámetros y retorna el resultado con la sentencia *return*.
 - De manera similar con el cálculo del capacitor en serie, crear otro método estático para que calcule y retorne el valor del capacitor equivalente en paralelo.
3. **(40%)** Una estación de venta de combustibles almacena el Diesel y de Gasolina a vender en 2 tanques subterráneos idénticos (ver diagrama); un tanque diferente por cada tipo de combustible.

Cada tanque es cilíndrico y sus medidas están dadas en metros y la densidad de la gasolina adquirida es de 0.750 g/mL.

La aplicación en Java debe solicitar las medidas de los tanques, para así determinar el volumen máximo (en Litros) de Diesel que puede ser almacenado en el 1er tanque.

Luego, del 2do tanque, se requiere determinar la cantidad límite de Gasolina (en Kilogramos) que puede ser almacenada.

Restricciones de diseño:

- Clase implementada para la solución no debe incluir campos, solamente define métodos de acceso estático. Cada campo declarado ocasionará un descuento del 30% en la nota de la solución.
- La función **main** nunca dialoga con usuario (imprime o solicita valores en pantalla), solo almacena los valores de entrada del problema y los valores de salida esperados de la solución. Nunca realiza operaciones de cálculo, solo invoca a otros métodos.
- El valor brindado sobre la gasolina se considera constante, por lo que debe ser declarado como tal en el código fuente y utilizarlo cuando se requiera.
- Por cada dato de entrada distinto, identificado del ejercicio, se elabora un método para solicitarlo a usuario y retornarlo (con sentencia *return*).
- La impresión de los datos generados de salida se realiza en un único método.



VI. REFERENCIA BIBLIOGRAFICA.

Libros en la biblioteca de la UDB

- Fain, Yakov. 2011. Programación Java. Madrid: Anaya Multimedia.
- Vegas Gertrudix, J. M. (2022). Java 17 : programación avanzada. Ra-Ma. <http://www.digitaliapublishing.com/a/116382/>
- Groussard, T. (2013). Java 7 : bases del lenguaje y de la programación orientada a objetos. ENI.